Administrador: objetos de base de datos para quitar las consultas directas de los DAO

Zaca - Terminal de autobuses | 26 de octubre de 2026 | Alcance: solo la seccion del administrador, contrastado con el esquema real (PostgreSQL 16.9). (Cuentas, Personal, Taquillas, Unidades, Rutas y el alta rapida de ciudad).

1. Resumen

Hoy los DAO del administrador arman SQL de tablas dentro de Java: **32 sentencias** repartidas en 6 clases. El objetivo es que el DAO solo haga **SELECT sobre una vista**, **SELECT sobre una funcion de lectura** o **CALL a un procedure**, y que la aplicacion ya no tenga permiso de INSERT/UPDATE/DELETE directo sobre las tablas. Se sigue la regla del contrato de datos: vista para consultas armadas, funcion solo para lecturas con parametros, procedure para todo lo que modifica datos (por Pgpool), trigger para validaciones automaticas.

Tipo de objeto	Cantidad	Para que
Vistas	13	Reemplazan los 13 SELECT de listados y catalogos (rutas se arma con 3 vistas).
Procedures	9	Reemplazan los INSERT / UPDATE / DELETE: un CALL por operacion, con su transaccion dentro.
Funciones	2	Lecturas con parametro que hoy se resuelven en Java (usuario sugerido y motivos para no eliminar).
Triggers	7	Reglas que hoy solo valida el formulario JavaFX o que nadie valida; la base debe ser la ultima defensa.
Constraints / indices	1 nuevo	Casi todos ya existen en la base real (se consulto el esquema vivo); solo falta UNIQUE sobre lower(nombre_terminal).

Convenciones propuestas

Nombres: vistas **v_***, procedures **sp_***, funciones **fn_***, triggers **trg_***, esquemas **administracion** y **viajes**. Los ids que se generan salen por un parametro **INOUT** (el procedure no devuelve filas). Los errores de regla usan **RAISE EXCEPTION ... USING ERRCODE** con los codigos Z del contrato; los codigos nuevos propuestos estan marcados como *(nuevo)* y falta confirmarlos con el equipo. Permisos: al usuario de la aplicacion solo **GRANT SELECT** sobre las vistas y **GRANT EXECUTE** sobre procedures y funciones; **REVOKE** de INSERT/UPDATE/DELETE sobre las tablas. Eso es lo que hace que ya no existan consultas directas.

2. Inventario de consultas directas actuales y su reemplazo

DAO	Metodo	Consulta directa de hoy	Se reemplaza por
CuentaDAO	roles()	SELECT administracion.rol	`v_roles`
CuentaDAO	listar()	SELECT con 5 tablas (personal_app, personal, rol, terminal, taquilla)	`v_cuentas`
CuentaDAO	guardar() alta	INSERT personal_app	`sp_guardar_cuenta`
CuentaDAO	guardar() edicion	UPDATE personal_app (rol, sede, taquilla, activo) + UPDATE aparte para el hash	`sp_guardar_cuenta`
CuentaDAO	cambiarEstado()	UPDATE personal_app SET activo	`sp_cambiar_estado_cuenta`
PersonalDAO	puestos()	SELECT administracion.puesto	`v_puestos`
PersonalDAO	listar()	SELECT personal + puesto + EXISTS(personal_app)	`v_personal`
PersonalDAO	guardar() alta y edicion	INSERT / UPDATE personal	`sp_guardar_personal`
PersonalDAO	eliminar()	DELETE personal (cascada a cuenta)	`sp_eliminar_personal` + `fn_motivos_no_eliminar`
TaquillaDAO	listar()	SELECT taquilla + terminal	`v_taquillas`
TaquillaDAO	guardar() alta y edicion	INSERT / UPDATE taquilla	`sp_guardar_taquilla`
UnidadDAO	tipos()	SELECT viajes.nombre_tipo_bus	`v_nombres_tipo_bus`
UnidadDAO	acomodos()	SELECT viajes.layout	`v_acomodos`
UnidadDAO	listar()	SELECT unidad + tipo_bus + nombre_tipo_bus + layout	`v_unidades`
UnidadDAO	guardar()	INSERT / UPDATE unidad	`sp_guardar_unidad`
UnidadDAO	buscarOCrearTipoBus()	SELECT y luego INSERT tipo_bus (combinacion tipo, acomodo, capacidad)	`sp_asegurar_tipo_bus`
RutaDAO	servicios()	SELECT viajes.tipo_servicio	`v_servicios`
RutaDAO	listar() - paradas	SELECT ruta_paradas + terminal	`v_ruta_paradas`
RutaDAO	listar() - precios	SELECT precio_tramo	`v_ruta_precios`
RutaDAO	listar() - rutas	SELECT ruta + tipo_servicio	`v_rutas`
RutaDAO	guardar()	INSERT/UPDATE ruta, DELETE + INSERT de paradas, INSERT de precios (5 sentencias en una transaccion)	`sp_guardar_ruta`
TerminalDAO	listar()	SELECT administracion.terminal	`v_terminales`
TerminalDAO	crear()	INSERT terminal con subconsulta de estado_conexion	`sp_crear_terminal`

Fuera de este documento: `TerminalDAO.obtenerDestinosValidos` (panel Destino del taquillero) y los DAO del despachador y del taquillero. Los que reciben ids por parametros de sesion (por ejemplo la sede) se veran en su propio documento.

3. Lo que se encontro en la base real

Antes de definir los objetos se consulto el esquema vivo (columnas, constraints, vistas, funciones, triggers y catalogos). Esto corrige o precisa lo que se asumia desde el contrato.

Hallazgo	Efecto en el diseno
Todos los ids son **GENERATED ALWAYS AS IDENTITY**	Un INSERT no puede mandar el id. Las altas devuelven el id nuevo por un parametro INOUT (RETURNING dentro del procedure).
La aplicacion se conecta como **postgres** (superusuario)	Un REVOKE no tendria efecto. Hay que crear un rol de aplicacion (por ejemplo **zaca_app**) con solo SELECT en vistas y EXECUTE en procedures/funciones, y cambiar DB_USER y DB_PASSWORD en el .env. Como ese rol no tendra DML, los procedures deben ser **SECURITY DEFINER** con **SET search_path** fijo, o fallaran con permiso denegado.
Ya existen casi todos los constraints	CHECK de anio (> 1900 y <= anio actual + 1), km >= 0, capacidad > 0, rango_inicial <= rango_final, CURP con sexo H o M, telefono de 10 digitos, km y min >= 0 en paradas, precio >= 0, orden_origen < orden_destino; UNIQUE de username, id_personal en personal_app, placa, identificador, serie_folio, curp, (id_ruta, orden) y (id_ruta, id_terminal). **Por eso se elimina el trigger trg_unidad_anio** que se habia propuesto.
Faltan estas reglas	UNIQUE sobre lower(nombre_terminal); licencia y vigencia obligatorias para CONDUCTOR (no hay CHECK ni trigger); folios emitidos; taquilla coherente con la sede y el rol; primera parada en 0 y km/min crecientes; precios completos; ruta en uso; empleado no eliminable.
precio_tramo ya permite 0	El CHECK real es precio >= 0, igual que el formulario de Rutas. El contrato dice > 0: hay que actualizar el contrato o la base (ver decisiones).
Solo hay triggers en ventas y viajes	tigresito_preparar_boleto, tigresito_validar_folio y tigresito_validad_conductor. Ninguna tabla de administrador tiene triggers.
Observacion fuera del alcance del administrador	En **viajes.fn_asignar_conductor** el bloque que debe lanzar Z0006 (licencia vencida) esta vacio: el IF no hace RAISE. Hoy una licencia vencida no se rechaza al programar un viaje. Se avisa porque depende de los datos de licencia que captura Personal.
La vista **administracion.v_directorio_usuarios** ya existe	Trae usuario, nombre, rol, puesto, caja y ubicacion, sin ids, y la ubicacion sale de la taquilla y no de personal_app.id_terminal. No sirve para Cuentas; se crea **v_cuentas** aparte.
Catalogos que el codigo busca por nombre	rol: ADMINISTRADOR, DESPACHADOR, TAQUILLERO. puesto: CONDUCTOR, TAQUILLERO, DESPACHADOR, MECANICO, ADMINISTRADOR. estado_conexion: DESCONECTADO (id 4). tipo_servicio: DIRECTO, ORDINARIO. layout: 2+2, 2+1, 1+1. nombre_tipo_bus: PRIMERA, EJECUTIVO, ORDINARIO. Los procedures y las vistas no deben cambiar estos nombres: el codigo Java compara con TAQUILLERO y CONDUCTOR, arma los paneles de Personal con los nombres de puesto y TerminalDAO busca DESCONECTADO.
tipo_bus ya tiene una fila	Id 5: PRIMERA, 2+2, 60 asientos. Las combinaciones nuevas se siguen creando al usarse, asi que sp_asegurar_tipo_bus sigue haciendo falta.
terminal: direccion e id_conexion son NOT NULL sin valor por defecto	sp_crear_terminal debe recibir una direccion no vacia y poner id_conexion = DESCONECTADO por su cuenta.
Extensiones	btree_gist y plpgsql. Version 16.9: los procedures con parametros INOUT son compatibles.

4. Vistas (13): columnas exactas

Regla para que no falle: los DAO leen el `ResultSet` por **posicion** (`rs.getInt(7)`), no por nombre. Cada vista debe devolver las columnas **en este orden y con este tipo**; cualquier columna extra va **al final**. Las vistas no garantizan orden de filas, asi que cada DAO conserva su `ORDER BY` al hacer SELECT sobre la vista.

Catalogos de dos columnas

Los usa `Sql.catalogo()`, que lee `getInt(1)` y `getString(2)`: las dos primeras columnas deben ser id (integer) y nombre (text/varchar), en ese orden.

Vista	Col. 1 (integer, NOT NULL)	Col. 2 (texto, NOT NULL)	ORDER BY del DAO	Extra al final
`administracion.v_roles`	id_rol	nombre_rol	id_rol	-
`administracion.v_puestos`	id_puesto	nombre_puesto	id_puesto	-
`administracion.v_terminales`	id_terminal	nombre_terminal	nombre_terminal	direccion (text)
`viajes.v_nombres_tipo_bus`	id_nombre_tipo_bus	nombre_tipo_bus	id_nombre_tipo_bus	-
`viajes.v_acomodos`	id_layout	nombre_layout (varchar(3), ej. 2+2)	id_layout	-
`viajes.v_servicios`	id_servicio	nombre_servicio	id_servicio	-

`administracion.v_cuentas`

Lista de Cuentas. ORDER BY username. Sede y taquilla pueden ser nulas (LEFT JOIN). **Cada par id + nombre debe quedar contiguo** porque el DAO lee el nombre en la columna siguiente al id.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_app`	integer	NO	getInt(1)
2	`id_personal`	integer	NO	getInt(2)
3	`username`	text	NO	getString(3)
4	`nombre_completo`	text	NO	getString(4): nombre_personal || ' ' || ap_paterno || ' ' || ap_materno
5	`id_rol`	integer	NO	getInt(5)
6	`nombre_rol`	text	NO	getString(6)
7	`id_terminal`	integer	SI	opcional(rs, 7): getObject(7, Integer.class); null = sin sede
8	`nombre_terminal`	text	SI	getString(8), solo si el id no es null
9	`id_taquilla`	integer	SI	opcional(rs, 9)
10	`identificador`	text	SI	getString(10), solo si el id no es null
11	`activo`	boolean	NO	getBoolean(11)

administracion.v_personal

Rejilla de Personal. ORDER BY ap_paterno, ap_materno, nombre_personal.

Pos	Columna	Tipo	NULL	Como la lee Java
1	id_personal	integer	NO	getInt(1)
2	id_puesto	integer	NO	getInt(2)
3	nombre_puesto	text	NO	getString(3): el codigo agrupa paneles por este nombre
4	nombre_personal	text	NO	getString(4)
5	ap_paterno	text	NO	getString(5)
6	ap_materno	text	NO	getString(6)
7	curp	text	NO	getString(7)
8	telefono	varchar(10)	SI	getString(8)
9	licencia	text	SI	getString(9)
10	vigencia	date	SI	getObject(10, LocalDate.class)
11	tiene_cuenta	boolean	NO	getBoolean(11): EXISTS sobre personal_app; no puede salir NULL

administracion.v_taquillas

Tabla de Taquillas. ORDER BY nombre_terminal, identificador.

Pos	Columna	Tipo	NULL	Como la lee Java
1	id_taquilla	integer	NO	getInt(1)
2	id_terminal	integer	NO	getInt(2)
3	nombre_terminal	text	NO	getString(3)
4	identificador	text	NO	getString(4)
5	serie_folio	text	NO	getString(5)
6	rango_inicial	integer	NO	getInt(6)
7	rango_final	integer	NO	getInt(7)
8	folios_usados	integer	NO	getInt(8)
9	activo	boolean	NO	getBoolean(9)

viajes.v_unidades

Tabla de Unidades. ORDER BY placa. **id_nombre_tipo_bus** es el tipo de servicio del autobus y **id_layout** el acomodo; no confundir con viajes.tipo_bus.id_tipo_bus.

Pos	Columna	Tipo	NULL	Como la lee Java
1	id_unidad	integer	NO	getInt(1)
2	placa	text	NO	getString(2)
3	modelo	text	NO	getString(3)
4	anio	integer	NO	getInt(4)
5	kilometraje	integer	NO	getInt(5)
6	id_nombre_tipo_bus	integer	NO	getInt(6)
7	nombre_tipo_bus	text	NO	getString(7)
8	id_layout	integer	NO	getInt(8)
9	nombre_layout	varchar(3)	NO	getString(9)
10	capacidad	integer	NO	getInt(10): viene de tipo_bus
11	en_mantenimiento (extra)	boolean	NO	Java no la lee hoy; va al final. false hasta que exista el FDW a taller_db

viajes.v_rutas

Cabecera de ruta. ORDER BY nombre_ruta.

Pos	Columna	Tipo	NULL	Como la lee Java
1	id_ruta	integer	NO	getInt(1)
2	nombre_ruta	text	NO	getString(2)
3	id_servicio	integer	NO	getInt(3)
4	nombre_servicio	text	NO	getString(4)
5	en_uso (extra)	boolean	NO	No se lee hoy; true si hay viajes de esa ruta

viajes.v_ruta_paradas

Paradas. ORDER BY id_ruta, orden. **El orden de las columnas difiere de la tabla**: km va antes que min, igual que la consulta actual de RutaDAO.

Pos	Columna	Tipo	NULL	Como la lee Java
1	id_ruta	integer	NO	getInt(1): clave del mapa de paradas
2	id_terminal	integer	NO	getInt(2)
3	nombre_terminal	text	NO	getString(3)
4	km_desde_origen	integer	NO	getInt(4) -> Parada.km
5	min_desde_origen	integer	NO	getInt(5) -> Parada.min
6	orden (extra)	integer	NO	No se lee; la posicion en pantalla sale del ORDER BY. Empieza en 1

viajes.v_ruta_precios

Precio de cada tramo. Sin ORDER BY en el DAO.

Pos	Columna	Tipo	NULL	Como la lee Java
1	id_ruta	integer	NO	getInt(1)
2	orden_origen	integer	NO	getInt(2): posicion desde 1, no desde 0
3	orden_destino	integer	NO	getInt(3)
4	precio	numeric	NO	getBigDecimal(4) -> Tramo.precio

5. Procedures (9): parametros exactos

Todos **LANGUAGE plpgsql SECURITY DEFINER SET search_path = administracion, viajes, pg_temp**. Un id INOUT entra NULL (o 0) en una alta y sale con el id generado; en una edicion entra con el id y sale igual. Los NULL de Java deben mandarse con `setNull(i, Types.X)`, no como cadena vacia: los campos opcionales (sede, taquilla, telefono, licencia, vigencia) se guardan NULL, nunca ''.

`administracion.sp_guardar_cuenta`

Sustituye el INSERT, los dos UPDATE y la transaccion de CuentaDAO.guardar().

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_app (INOUT)	integer	SI	Cuenta.idApp(): 0 = alta, se manda NULL
2	p_id_personal	integer	NO	Cuenta.idPersonal() (solo se usa en alta)
3	p_id_rol	integer	NO	Cuenta.rol().id()
4	p_id_terminal	integer	SI	Cuenta.sede() == null ? NULL : sede().id()
5	p_id_taquilla	integer	SI	Cuenta.taquilla() == null ? NULL : taquilla().id()
6	p_username	text	NO	Cuenta.username(). En edicion se ignora. Si llega NULL en alta, el procedure usa fn_proponer_username
7	p_password_hash	text	SI	Hash bcrypt de 60 caracteres. Obligatorio en alta; NULL en edicion = no cambia la contrasena
8	p_activo	boolean	NO	Cuenta.activa()

`administracion.sp_cambiar_estado_cuenta`

Sustituye CuentaDAO.cambiarEstado().

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_app	integer	NO	idApp
2	p_activo	boolean	NO	activa
3	p_id_actor	integer	SI	Cuenta en sesion (Sesion.actual().idApp()); NULL mientras el login sea de muestra

`administracion.sp_guardar_personal`

Sustituye PersonalDAO.guardar().

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_personal (INOUT)	integer	SI	Empleado.idPersonal(): 0 = alta, se manda NULL
2	p_id_puesto	integer	NO	puesto().id()
3	p_nombre	text	NO	nombre()
4	p_ap_paterno	text	NO	apPaterno()
5	p_ap_materno	text	NO	apMaterno()
6	p_curp	text	NO	En mayusculas, 18 caracteres; el CHECK exige ^[A-Z]{4}[0-9]{6}[HM][A-Z]{5}[A-Z0-9][0-9]$
7	p_telefono	varchar(10)	SI	NULL o exactamente 10 digitos (el controlador manda NULL si esta vacio)
8	p_licencia	text	SI	Solo CONDUCTOR; NULL para los demas puestos
9	p_vigencia	date	SI	setObject(i, LocalDate); solo CONDUCTOR

`administracion.sp_eliminar_personal`

Sustituye PersonalDAO.eliminar().

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_personal	integer	NO	idPersonal
2	p_id_actor	integer	SI	Cuenta en sesion; NULL mientras no haya login funcional

`administracion.sp_guardar_taquilla`

Sustituye TaquillaDAO.guardar(). folios_usados no se envia: solo lo mueve la venta.

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_taquilla (INOUT)	integer	SI	Taquilla.idTaquilla(): 0 = alta, se manda NULL
2	p_id_terminal	integer	NO	terminal().id()
3	p_identificador	text	NO	identificador()
4	p_serie_folio	text	NO	serie()
5	p_rango_inicial	integer	NO	rangoInicial()
6	p_rango_final	integer	NO	rangoFinal()
7	p_activo	boolean	NO	activa()

`administracion.sp_crear_terminal`

Sustituye TerminalDAO.crear(). Pone id_conexion = DESCONECTADO (id 4 hoy) buscandolo por nombre.

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_terminal (INOUT)	integer	SI	Entra NULL; sale el id. Java lo convierte en new Catalogo(id, nombre)
2	p_nombre	text	NO	Nombre de la ciudad (unico sin importar mayusculas)
3	p_direccion	text	NO	NOT NULL en la tabla: el formulario ya la exige

`viajes.sp_asegurar_tipo_bus`

Sustituye UnidadDAO.buscarOCrearTipoBus(). La llama sp_guardar_unidad; no la usa Java directamente.

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_tipo_bus (INOUT)	integer	SI	Entra NULL; sale el id existente o el recien creado
2	p_id_nombre_tipo_bus	integer	NO	Unidad.tipo().id() (tipo de servicio: PRIMERA, EJECUTIVO, ORDINARIO)
3	p_id_layout	integer	NO	Unidad.acomodo().id()
4	p_capacidad	integer	NO	Unidad.capacidad(), > 0

`viajes.sp_guardar_unidad`

Sustituye UnidadDAO.guardar().

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_unidad (INOUT)	integer	SI	Unidad.idUnidad(): 0 = alta, se manda NULL
2	p_id_nombre_tipo_bus	integer	NO	tipo().id()
3	p_id_layout	integer	NO	acomodo().id()
4	p_capacidad	integer	NO	capacidad()
5	p_placa	text	NO	placa()
6	p_modelo	text	NO	modelo()
7	p_anio	integer	NO	anio(): el CHECK real exige > 1900 y <= anio actual + 1
8	p_kilometraje	integer	NO	km(), >= 0

`viajes.sp_guardar_ruta`

Sustituye las 5 sentencias de RutaDAO.guardar().

#	Parametro	Tipo	NULL	Valor que manda Java
1	p_id_ruta (INOUT)	integer	SI	Ruta.idRuta(): 0 = alta, se manda NULL
2	p_id_servicio	integer	NO	servicio().id()
3	p_nombre	text	NO	nombre()
4	p_paradas	jsonb	NO	Arreglo en orden: [{"id_terminal":3,"km":0,"min":0}, ...]. El orden 1..n es la posicion. Java: setString + marcador ?::jsonb
5	p_precios	jsonb	NO	[{"orden_origen":1,"orden_destino":2,"precio":150.00}, ...]. Posiciones desde 1; precio como numero, no cadena

6. Funciones (2)

Funcion	Parametros	Devuelve	Reemplaza
`administracion.fn_proponer_username`	p_id_personal integer	text (inicial del nombre + apellido paterno, sin acentos, minusculas, con numero si se repite: m.constantino, m.constantino2)	NombreUsuario en Java. sp_guardar_cuenta la usa cuando p_username llega NULL; el UNIQUE de username sigue siendo la garantia final.
`administracion.fn_motivos_no_eliminar`	p_id_personal integer	TABLE(motivo text). Sin filas = se puede eliminar	El aviso que hoy solo aparece cuando falla el DELETE (23503). Se lee con SELECT motivo FROM fn_motivos_no_eliminar(?).

7. Triggers (7) y constraints

Todos lanzan el error con `RAISE EXCEPTION ... USING ERRCODE` para que Java lea `e.getSQLState()`. **Ninguna tabla de administrador tiene triggers hoy.** Se quito trg_unidad_anio porque check_anio_futuro ya cubre esa regla.

Trigger	Tabla y momento	Regla	Error	Hoy se valida en
`trg_personal_licencia`	BEFORE INSERT/UPDATE en personal	Puesto CONDUCTOR: licencia y vigencia no pueden ser NULL.	23514	Solo PersonalController (los campos de licencia). Hoy un INSERT directo lo permite.
`trg_personal_no_eliminable`	BEFORE DELETE en personal	Rechaza borrar a un chofer con viajes PROGRAMADO, ABORDANDO o EN RUTA.	Z0011	Solo la FK fk_conductor_viaje (23503), que ademas bloquea cualquier viaje, incluso cerrado.
`trg_cuenta_validar`	BEFORE INSERT/UPDATE en personal_app	La taquilla solo con rol TAQUILLERO, y debe ser de la misma terminal que la sede de la cuenta.	23514	CuentasController filtra el combo; la base no lo exige.
`trg_taquilla_folios`	BEFORE UPDATE en taquilla	Con folios_usados > 0 no se cambia serie_folio ni rango_inicial, y rango_final debe cubrir los usados.	Z0009	Nadie lo valida.
`trg_ruta_paradas_validar`	CONSTRAINT TRIGGER diferido en ruta_paradas	Al menos 2 paradas; orden 1..n sin huecos; primera parada en km 0 y minuto 0; km y minutos crecientes.	23514	RutasController valida >= 0, 2 paradas y terminal unica; no valida el resto (ver decision 2).
`trg_ruta_paradas_en_uso`	BEFORE INSERT/UPDATE/DELETE en ruta_paradas	Con viajes en la ruta no se agregan, quitan ni reordenan paradas; km y minutos si se pueden cambiar.	Z0008	Nadie lo valida; RutaDAO.guardar borra y reinserta todo.
`trg_precio_tramo_completo`	CONSTRAINT TRIGGER diferido en ruta y precio_tramo	Hay precio para cada par origen < destino de las paradas, y ninguno fuera de rango.	23514	RutasController obliga a escribir todos los precios.

Constraints: estado real

Constraint	Tabla	Estado	Para que
UNIQUE (lower(nombre_terminal))	terminal	**FALTA**	Ciudad unica sin importar mayusculas (hoy se pueden duplicar Tapachula y TAPACHULA).
unique_ruta_orden, unique_ruta_terminal	ruta_paradas	Existe	Sin orden repetido ni terminal repetida.
check_km_positivo, check_min_positivo	ruta_paradas	Existe	km y minutos >= 0.
check_precio_tramo (>= 0), check_tramo_orden	precio_tramo	Existe	El contrato dice > 0 (decision 1).
check_rangos, check_folios_positivo, UNIQUE identificador, UNIQUE serie_folio	taquilla	Existe	Rango coherente y llaves unicas.
UNIQUE username, UNIQUE id_personal	personal_app	Existe	Una cuenta por persona.
check_curp_valido (sexo H o M), check_telefono, UNIQUE curp	personal	Existe	Igual que el script V6/V7.
check_anio_pasado, check_anio_futuro, check_km_positivo, UNIQUE placa; check_capacidad_positivo	unidad, tipo_bus	Existe	Reglas de unidad.
fk_conductor_viaje RESTRICT; fk_personal_app CASCADE; fk_taquilla_app SET NULL; fk_terminal_app RESTRICT	varias	Existe	Eliminar personal borra su cuenta; terminal con cuentas no se borra.

8. Como llamarlos desde Java (probado contra la base)

Se creo un procedure temporal con un parametro INOUT, un jsonb y un texto, y se llamo con el driver actual (postgresql 42.7.4) contra la base real:

Prueba	Resultado	Conclusion
CALL nativo con PreparedStatement + executeQuery, id INOUT enviado NULL, otros NULL con setObject(null)	Devolvio una fila con el id generado (99)	**Es el patron a usar**: `prepareStatement("CALL esquema.sp_x(?, ?, ?::jsonb)")` y leer la fila con rs.getInt(1).
CallableStatement con {call sp_x(...)} y registerOutParameter	FALLA: "es un procedimiento"	No usar {call ...}: con la configuracion por defecto el driver lo trata como funcion.
Error lanzado con ERRCODE = 'Z0013' dentro del procedure	e.getSQLState() = Z0013	Los codigos Z llegan intactos; los controladores ya comparan SQLState.
Llamada dentro de setAutoCommit(false) y commit()	Con id existente devolvio el mismo id (5)	El procedure funciona igual dentro o fuera de transaccion; el DAO ya no necesita Sql.transaccion para estos casos.

Tipo Java	Tipo SQL	Como se manda
int / Integer	integer	setInt; si es opcional y vacio: setNull(i, Types.INTEGER). Id 0 de una alta: mandar NULL
String	text / varchar(10)	setString. Opcional vacio: setNull(i, Types.VARCHAR) (nunca "")
boolean	boolean	setBoolean
LocalDate	date	setObject(i, localDate); vacio: setNull(i, Types.DATE)
BigDecimal	numeric	setBigDecimal; dentro del jsonb va como numero
Listas de paradas y precios	jsonb	Armar el JSON en el DAO, setString y marcador ?::jsonb
Id generado	integer INOUT	executeQuery(); rs.next(); rs.getInt(1)

9. Cambios de Java que hay que hacer para que no falle

Donde	Hoy	Que hacer
TaquillasController.mensajeRepetido	Distingue serie de identificador con e.getMessage().contains("serie_folio"), que viene del nombre del constraint (taquilla_serie_folio_key).	El procedure **no debe atrapar ni traducir** el 23505: tiene que dejar subir el error original. Si no, el mensaje cambia y la serie repetida se mostrara como identificador repetido.
PersonalController (eliminar)	Solo reconoce 23503 para "tiene registros".	Agregar Z0011 (y Z0013 si se adopta) al mismo mensaje.
CuentasController y CuentaDAO	El DAO recibe el hash; el usuario se arma en el formulario.	Mantener ese reparto: el procedure recibe username y p_password_hash. Idem NULL del hash en edicion.
Modelos Ruta y Unidad	No tienen campos para en_uso ni en_mantenimiento.	Opcional. Las columnas extra van al final de la vista y Java las ignora hasta que los records las incluyan.
RutasController (decisiones 2 y 3)	Acepta paradas desordenadas y la primera parada distinta de 0.	Ajustar el formulario antes de activar trg_ruta_paradas_validar; si no, esas capturas daran 23514.
Sql.catalogo y los DAO de listado	Leen por posicion.	Sin cambios si las vistas respetan las posiciones de la seccion 4.
ConexionBD / .env	Usuario postgres.	Cambiar a zaca_app cuando se hagan los REVOKE (seccion 3).

10. Decisiones y discrepancias que el equipo debe resolver

#	Tema	Situacion	Que hay que decidir o hacer
1	Precio de tramo	La base real permite precio >= 0 y el formulario tambien; el contrato dice > 0.	Decidir si se acepta precio 0 (tramo gratis) y actualizar el contrato, o cambiar el CHECK y el formulario.
2	Km y minutos crecientes	El contrato pide primera parada en 0 y valores crecientes; la base solo exige >= 0 y el formulario tambien.	Con trg_ruta_paradas_validar se rechazarian capturas hoy validas. Ajustar el formulario primero.
3	Editar paradas de una ruta en uso	RutaDAO.guardar borra y reinserta todas las paradas (y por cascada los precios).	sp_guardar_ruta debe actualizar en sitio cuando la ruta esta en uso; el formulario debe bloquear terminales y orden si v_rutas.en_uso.
4	Borrado real o baja logica de personal	El contrato lo deja abierto.	Si se elige baja logica, sp_eliminar_personal pasa a sp_dar_baja_personal y v_personal agrega la columna activo.
5	Usuario en sesion	p_id_actor evita auto-desactivarse o auto-eliminarse.	Requiere login funcional y Sesion.iniciar(cuenta). Mientras tanto se pasa NULL y esa regla no se aplica.
6	Codigo Z0013	No existe en el contrato (llega hasta Z0012).	Aprobar ACCION_SOBRE_SI_MISMO o reutilizar otro.
7	Rol de aplicacion	Hoy se conecta postgres.	Crear zaca_app, mover las credenciales del .env y marcar los procedures SECURITY DEFINER. Sin esto el REVOKE no sirve.
8	Generacion del usuario	NombreUsuario en Java o fn_proponer_username en la base.	Se recomienda la base (sin carreras entre altas simultaneas); el formulario conserva la vista previa.
9	Licencia vencida (fuera del alcance)	El bloque Z0006 de fn_asignar_conductor esta vacio.	Avisar a quien lleve la parte de viajes.

11. Orden sugerido de implementacion

Paso	Que	Detalle
1	Rol zaca_app y UNIQUE lower(nombre_terminal)	Sin cambio de comportamiento. Antes del UNIQUE, revisar que no haya ciudades duplicadas.
2	Las 13 vistas	Solo lectura. Cada DAO cambia su SELECT por SELECT sobre la vista; los modelos no cambian si se respetan las posiciones.
3	Procedures, de simples a complejos	Terminal, taquilla, personal, cuenta, unidad (con sp_asegurar_tipo_bus) y al final sp_guardar_ruta. Un DAO a la vez, probando cada operacion contra la base.
4	Funciones	fn_motivos_no_eliminar primero (cambia el dialogo de eliminar), luego fn_proponer_username.
5	Triggers	Despues de ajustar los formularios (decisiones 2 y 3). Revisar antes que los datos actuales los cumplan.
6	REVOKE de DML directo	Ultimo paso: con la aplicacion usando solo CALL y SELECT y conectada como zaca_app.